Parallel Computing

Paper Implementation & Performance Analysis Project

Spring 2026 | 25% of Final Grade

Project Overview

In this project, your team will select a published research paper describing a parallelizable algorithm, implement it from scratch, and conduct a rigorous performance analysis comparing the serial and parallel versions.

Key Details

Team size: 2 students
Duration: 5 weeks
Weight: 25% of the final course grade
Parallelism framework: OpenMP (CPU) or CUDA (GPU)
Expected effort: ~10 person-hours total for implementation

What You Will Do

1. **Choose a paper** from the pool of papers allowed (multiple groups are allowed to take the same paper) or pick a paper of your choice (subject to instructor/TA approval).
2. **Implement a serial version** in C/C++. LLM coding tools are allowed for this step.
3. **Parallelize the code** using OpenMP or CUDA. LLM tools are **not permitted** for parallelization.
4. **Perform a detailed performance analysis** and write a final report. (LLM tools are not allowed for this step as well)

⚠ LLM Usage Policy

- ✅ **Serial implementation:** LLM tools allowed.
- ✅ **Paper understanding:** LLM tools allowed.
- ✅ **Compile error assistance (without the source code):** LLM tools allowed.
- ✅ **Test generation:** LLM tools allowed.
- ❌ **Parallel implementation:** LLM tools NOT allowed. Write all parallel code yourself.
- ❌ **Performance analysis/debugging:** LLM tools NOT allowed. Use Nsight/nvprof/perf etc..
- ❌ **Report writing:** Must be your own work.

Project Timeline

The project spans five weeks.

Week	Milestone	Details
Week 1-2	Paper Selection & Setup	Select paper from the pool provided. Read and understand the algorithm. Set up the development environment. Identify input datasets and define correctness tests.
Week 2-3	Serial Implementation	Implement a working serial (single-threaded) version in C/C++. You may use LLM-based coding tools to assist. Validate correctness against known results or small test cases.
Week 3-4	Parallelization & Presentation	Parallelize the serial code using OpenMP or CUDA. You must write the parallel code yourselves—LLM tools are NOT permitted. Verify correctness. Begin collecting performance data. Prepare a 5-minute in-class presentation.
Week 5	Analysis & Report	Conduct detailed performance analysis: speedup, strong/weak scaling, bottlenecks. Write and submit the final report.

Deliverables

- **Source code:** Clean, well-commented repo with serial and parallel implementations, Makefile, and README.
- **Final report (PDF, 3–6 pages):** Paper summary, implementation, parallelization strategy, performance results with graphs, bottleneck discussion, lessons learned. (Overleaf template will be provided later)
- **In-class presentation:** 6 minutes summary of paper, approach, and key results.

Grading Breakdown

Component	Weight
Parallel implementation (OpenMP or CUDA)	**25%**
Final report -- Performance analysis (speedup, scaling, bottleneck discussion, ablation study)	**25%**
Final report -- (clarity, depth, presentation of results)	**25%**
In-class presentation (5 min)	**25%**

Performance Analysis Expectations

- **Speedup plot:** Execution time and speedup vs. thread count or problem size.
- **Strong scaling: (https://hpc-wiki.info/hpc/Scaling)** Fix problem size, increase threads. Report on parallel efficiency.
- **Weak scaling** (if applicable) (https://hpc-wiki.info/hpc/Scaling) : Increase both problem size and threads proportionally.
- **Bottleneck analysis:** Memory bandwidth? Synchronization? Load imbalance? Use nsight systems profiling tools. (https://www.youtube.com/watch?v=dUDGO66IadU&list=PL5B692fm6--ukF8S7ul5NmceZhXLRv_IR&index=4)
- **Performance improvements:** To address bottlenecks, implement at least 3 performance optimizations. Show their incremental benefits through an ablation study.
- Compare your results with the paper's reported numbers.

Tips for Success

- **Start early.** Read the paper thoroughly in Week 1. Clarify any confusion with the instructor.
- **Get serial right first.** Debug parallel code against a correct serial baseline.
- **Think about the data layout.** Coalesced access, shared memory tiling, row/column major layout etc.
- **Parallelize incrementally.** Start with the hottest loop or kernel.
- **For LLM/Transformer papers:** Use random Q, K, V matrices as test inputs. No framework is needed.
- **Use Notebook LLM to better understand the papers.**

Paper Selection

- Below is an initial list of papers you could pick from. Please note that the implementations are not tested by us, and they are not guaranteed to provide the full performance improvements promised in the paper.
- If you prefer, you could find another paper published in an HPC/Architecture/Systems/ML top conference. If you choose this route, you need to check with the instructor/TA to make sure it is an appropriate paper.

Paper Pool — Quick Reference Table (clickable)

Click any paper name to open the publication. Detailed descriptions follow.

#	Algorithm / Paper	Venue, Year	Category	Platform
1	K-means clustering (Cuomo et al.)	Comp. & EE, 2018	AI/ML	CUDA
2	k-NN search (Sismanis et al.)	HPEC, 2012	AI/ML	CUDA
3	Naive Bayes / GPU-NB (Andrade et al.)	SBAC-PAD, 2013	AI/ML	CUDA
4	Logistic regression / SGD (Do & Poulet)	ACIIDS, 2015	AI/ML	OpenMP
5	Decision tree / ID3 (Nasridinov et al.)	Computing, 2014	AI/ML	CUDA
6	SVM multiclass hybrid (Tan et al.)	IEEE JSTARS, 2015	AI/ML	Both
7	Matrix factorization / CCD++ (Yu et al.)	KAIS, 2014	AI/ML	OpenMP
8	EM / GMM (Altinigneli et al.)	KDD, 2013	AI/ML	CUDA
9	Random forest / CudaTree (Liao et al.)	NIPS Wksp, 2013	AI/ML	CUDA
10	Approximate SVD / QUIC-SVD (Foster et al.)	PPAM, 2012	AI/ML	CUDA
11	Scaled dot-product attention (Vaswani et al.)	NeurIPS, 2017	LLM	CUDA
12	Online softmax (Milakov & Gimelshein)	arXiv, 2018	LLM	CUDA
13	Layer normalization kernel (Ba et al.)	arXiv, 2016	LLM	CUDA/OMP
14	Adaptive softmax (Grave et al.)	ICML, 2017	LLM	CUDA
15	Word2Vec SGNS on GPU (Li et al.)	HiPC, 2016	LLM	CUDA
16	Sparse Transformers / fixed patterns (Child et al.)	arXiv, 2019	LLM	CUDA
17	Multi-query attention (Shazeer)	arXiv, 2019	LLM	CUDA
18	Rotary position embedding / RoPE (Su et al.)	arXiv, 2021	LLM	CUDA
19	Linformer / projected attention (Wang et al.)	arXiv, 2020	LLM	CUDA
20	Reformer / LSH attention (Kitaev et al.)	ICLR, 2020	LLM	CUDA
21	Grouped-query attention / GQA (Ainslie et al.)	EMNLP, 2023	LLM	CUDA

22	2D Jacobi stencil (Cecilia et al.)	PARA, 2012	Scientific	CUDA/OMP
23	Median filter (Perrot et al.)	JSPS, 2014	Scientific	CUDA
24	Bitonic sort (Peters et al.)	PPAM, 2010	Scientific	CUDA
25	Histogram computation (Gómez-Luna et al.)	MVA, 2013	Scientific	CUDA/OMP
26	Lattice Boltzmann / D3Q19 (Obrecht et al.)	CMA, 2011	Scientific	CUDA
27	Hybrid radix sort (Stehle & Jacobsen)	SIGMOD, 2017	Scientific	CUDA
28	String matching / Rabin-Karp (Ashkiani et al.)	SPAA, 2016	Scientific	CUDA
29	Cuckoo hash table (Alcantara et al.)	GPU Gems Jade, 2012	Scientific	CUDA
30	Smith-Waterman alignment (Zhao & Sahni)	ICCABS, 2012	Scientific	CUDA/OMP
31	Optimized 3D stencil (Krotkiewski & Dąbrowski)	Parallel Comp., 2013	Scientific	CUDA
21	3D convex hull / CudaHull (Stein et al.)	Comp. & Graphics, 2012	Scientific	CUDA
33	BFS level-synchronous (Luo et al.)	DAC, 2010	Graph	CUDA
34	Community detection / Louvain (Forster)	arXiv, 2018	Graph	CUDA
35	Betweenness centrality (Sarıyüce et al.)	GPGPU-6, 2013	Graph	CUDA